SCD Lab Task 8

Defensive Programming and Silent Bug Detection

IMSCIENCES Peshawar

Course: Software Construction and Development Lab

Instructor: Engr. Muhammad Umer Haroon

Date: 29 October 2026

This report documents both tasks in the lab handout: reproducing crashes, guarding invalid inputs, identifying silent logic bugs, and verifying corrected behavior. All 57 required and additional checks passed against the patched functions.

Submission contents

crash_gallery.py contains the six original Task 1 functions, their patched versions and the executable test block. bug_hunt.py contains the six original Task 2 functions, corrected versions with explanatory comments and its executable test block. The original source is preserved verbatim in each module's ORIGINAL_SOURCE string and executed in a separate namespace so both versions can be tested in one run.

The accompanying output text files contain captured execution results. test_results.json records the expected, original and patched results. build_report.py regenerates the evidence and this report; Lab_Report.md is an editable text copy of the report.

Run instructions

Keep the two Python solution files together. Run python crash_gallery.py and then python bug_hunt.py from this folder. The solution programs require Python 3 and only the standard library; bug_hunt imports the numeric validation helper from crash_gallery. Rebuilding the PDF with python build_report.py additionally requires reportlab.

Input rules

Marks and passing thresholds use the inclusive 0 to 100 scale. Prices and monetary amounts must be nonnegative; discount values are percentages from 0 to 100. Tax rates are fractions, so 0.1 means 10 percent; no arbitrary upper tax-rate cap is imposed. Ages are nonnegative integers, student names use exact case-sensitive matching, and malformed student records produce clear errors. Boolean values, NaN and infinity are excluded from numeric input. parse_number accepts signed decimal integer text with optional surrounding whitespace; file reading uses UTF-8.

Error strings are returned for invalid input, except is_adult returns False and a missing student returns None, following the handout. Test harnesses catch original exceptions only so every case can run; patched functions use guard checks, with narrow handlers for unavoidable file I/O errors.

Task 1 The Crash Gallery

Parts A to C: original functions are preserved, failures below were observed by running them, and the same inputs were passed to the patched functions. Every function has at least three distinct bad-input cases plus a valid case. Error messages are recorded as produced by this Python runtime; operating-system wording can vary. Temporary paths in file errors are normalized to readable labels.

divide

Input	Original outcome	Error type	Patched result
divide(10, 0)	division by zero	ZeroDivisionError	'Error: cannot divide by zero'
divide('10', 2)	unsupported operand type(s) for /: ' str' and 'int'	TypeError	'Error: both arguments must be finite numbers'
divide(10, None)	unsupported operand type(s) for /: ' int' and 'NoneType'	TypeError	'Error: both arguments must be finite numbers'
divide(10, 2)	5.0	None	5.0

Patch: Validate both numeric arguments before division and reject a zero divisor.

get_first

Input	Original outcome	Error type	Patched result
get_first([])	list index out of range	IndexError	'Error: list is empty'
get_first(None)	'NoneType' object is not subscriptable	TypeError	'Error: input is not a list'
get_first(42)	'int' object is not subscriptable	TypeError	'Error: input is not a list'
get_first([7, 8])	7	None	7

Patch: Require a list and check that it is nonempty before indexing.

Task 1 The Crash Gallery continued

get_age

Input	Original outcome	Error type	Patched result
get_age({})	'age'	KeyError	'Error: missing age'
get_age('Ali')	string indices must be integers, not 'str'	TypeError	'Error: input is not a dictionary'
get_age(None)	'NoneType' object is not subscriptable	TypeError	'Error: input is not a dictionary'
get_age({'age': -1})	-1	None	'Error: age must be a nonnegative integer'
get_age({'age': 20})	20	None	20

Patch: Require a dictionary, verify the age key and reject invalid age values.

parse_number

Input	Original outcome	Error type	Patched result
parse_number('abc')	invalid literal for int() with base 10: ' abc'	ValueError	'Error: not a number'
parse_number(None)	int() argument must be a string, a bytes-like object or a real number, not 'NoneType'	TypeError	'Error: input must be text'
parse_number('3.5')	invalid literal for int() with base 10: ' 3.5'	ValueError	'Error: not a number'
parse_number('')	invalid literal for int() with base 10: ' '	ValueError	'Error: not a number'
parse_number(' -12 ')	-12	None	-12

Patch: Validate text, an optional sign and ASCII digits before int conversion; reject decimal fractions and oversized conversion strings.

Task 1 The Crash Gallery continued

read_file

Input	Original outcome	Error type	Patched result
read_file('missing.txt')	[Errno 2] No such file or directory: ' missing.txt'	FileNotFoundError	'Error: file not found'
read_file(None)	expected str, bytes or os.PathLike object, not NoneType	TypeError	'Error: filename must be nonempty text without null characters'
read_file('<directory>')	[Errno 13] Permission denied: ' <directory>'	PermissionError	'Error: path is not a regular file'
read_file('sample.txt')	'Hello, lab!'	None	'Hello, lab!'

Patch: Validate the filename, existence and regular-file status. A context manager closes the file even on failure; narrow handlers cover read-time operating-system and encoding errors.

average

Input	Original outcome	Error type	Patched result
average([])	division by zero	ZeroDivisionError	'Error: cannot average an empty list'
average(['a', 'b'])	unsupported operand type(s) for +: 'int' and 'str'	TypeError	'Error: every item must be a finite number'
average(None)	'NoneType' object is not iterable	TypeError	'Error: input is not a list'
average([10, 20, 30])	20.0	None	20.0

Patch: Require a nonempty list and verify every element before summing and dividing.

Task 1 Discussion Notes

Part D

Which function was hardest to make safe and why

read_file was the hardest because its success depends on the operating system as well as the input. A valid string can name a missing file, a directory, an unreadable file or text with an incompatible encoding. Checks handle common invalid paths, but a file can disappear or its permissions can change between checking and opening it, so narrow OSError and UnicodeError handlers protect the actual read.

Which kind of bad input was most common

Wrong types were the most common issue across the functions. Examples include text in arithmetic, None where a collection is required, and a string instead of a person dictionary. Empty collections and missing values were also recurring problems.

What pattern appeared in the fixes

Each fix validates the input type before using an operation that assumes that type, then checks required contents or allowed values. Invalid input returns a descriptive error early. Only validated input reaches the original calculation or lookup.

Did any function need more than one check and why

Yes. divide checks numeric types and a nonzero divisor; get_first checks for a list and then for an element; get_age checks the dictionary, the age key and the age value. average checks the list, its length and each number, while parse_number checks text, sign, digits and Python's conversion length limit. One successful check does not guarantee that every later operation is safe.

Verification: 26 of 26 Task 1 patched checks passed, including 20 bad-input cases. No patched call in this test set raised an exception.

Task 2 The Silent Bug Hunt

Parts A to C: each function was tested with at least three cases including boundaries. The Expected column defines the corrected contract; Bug indicates whether the original differs from that contract or crashes. Each fixed output matched Expected exactly. The corrected source includes a Bug comment and a Fix comment for every function.

calculate_discount

Test input	Expected and fixed	Actual original result	Bug
calculate_discount(100, 0)	100.0	100.0	No
calculate_discount(100, 150)	'Error: discount must be between 0 and 100'	-50.0	Yes
calculate_discount(100, 100)	0.0	0.0	No
calculate_discount(100, -10)	'Error: discount must be between 0 and 100'	110.0	Yes
calculate_discount(-100, 10)	'Error: price cannot be negative'	-90.0	Yes
calculate_discount(200, 25)	150.0	150.0	No

calculate_average

Test input	Expected and fixed	Actual original result	Bug
calculate_average([])	'Error: cannot average empty list'	ZeroDivisionError: division by zero	Yes
calculate_average([0, 100])	50.0	50.0	No
calculate_average([80, 90, 100])	90.0	90.0	No
calculate_average([-10, 90])	'Error: marks must be numbers between 0 and 100'	40.0	Yes
calculate_average([101])	'Error: marks must be numbers between 0 and 100'	101.0	Yes

Task 2 The Silent Bug Hunt continued

For the search tests, students = [{'name': 'Ali', 'age': 20}, {'name': 'Sara', 'age': 19}].

find_student

Test input	Expected and fixed	Actual original result	Bug
find_student(students, 'missing')	None	{'name': 'Ali', 'age': 20}	Yes
find_student([], 'Ali')	None	IndexError: list index out of range	Yes
find_student(students, 'Sara')	{'name': 'Sara', 'age': 19}	{'name': 'Sara', 'age': 19}	No
find_student([{}], 'Ali')	'Error: every student must have a nonempty text name'	KeyError: 'name'	Yes

is_adult

Test input	Expected and fixed	Actual original result	Bug
is_adult(18)	True	False	Yes
is_adult(17)	False	False	No
is_adult(19)	True	True	No
is_adult(-1)	False	False	No
is_adult('18')	False	TypeError: '>' not supported between instances of 'str' and 'int'	Yes

Task 2 The Silent Bug Hunt continued

apply_tax

Test input	Expected and fixed	Actual original result	Bug
apply_tax(100, -0.1)	'Error: tax rate cannot be negative'	90.0	Yes
apply_tax(100, 0)	100	100	No
apply_tax(100, 0.1)	110.0	110.0	No
apply_tax(-100, 0.1)	'Error: amount cannot be negative'	-110.0	Yes
apply_tax(0, 0.1)	0.0	0.0	No

count_passed

Test input	Expected and fixed	Actual original result	Bug
count_passed([50, 50, 50])	3	0	Yes
count_passed([49, 50, 51])	2	1	Yes
count_passed([])	0	0	No
count_passed([59, 60, 61], 60)	2	1	Yes
count_passed([-1, 50])	'Error: marks must be numbers between 0 and 100'	0	Yes
count_passed([50], 101)	'Error: passing mark must be between 0 and 100'	0	Yes

Task 2 Bug Report

Part D. Each entry states the intended behavior, observed defect and correction in three sentences.

calculate_discount

The function should calculate a reduced price from a nonnegative price and a discount between 0 and 100 percent. The original accepted 150 percent and returned -50.0 for a price of 100, and a negative discount increased the price. The fix validates finite numeric inputs, rejects negative prices and enforces the inclusive discount range before calculating the result.

calculate_average

The function should return the mean of a nonempty list of valid marks. The original divided by zero for an empty list and silently averaged invalid marks such as -10 or 101. The fix rejects empty or non-list input and requires every mark to be a finite number between 0 and 100 before computing the mean.

find_student

The function should return the matching student or None when the name is absent. The original returned the first student for a missing name, raised IndexError on an empty list and raised KeyError on a record without a name. The fix validates the records, returns an exact matching record and returns None after an unsuccessful search.

is_adult

The function should classify a valid integer age of 18 or above as adult. The original used a strict greater-than comparison and returned False for age 18. The fix rejects invalid or negative ages and uses age >= 18 so the threshold is included.

apply_tax

The function should add tax using a nonnegative amount and a nonnegative fractional tax rate. The original accepted a negative rate and returned 90.0 for amount 100 and rate -0.1, silently reducing the amount. The fix validates finite numeric inputs and rejects negative amounts and rates before applying the unchanged tax formula.

count_passed

The function should count marks equal to or greater than the passing threshold. The original used a strict comparison and returned zero for three marks of 50 at the default threshold of 50. The fix validates marks and the threshold and uses >= so scores exactly at the passing mark count as passes.

Task 2 Discussion

Which bug was hardest to find and why

The find_student fallback was the hardest to notice because it returned a well-formed student dictionary even when the requested name was absent. A test that only checks whether the result looks like a student can pass. Checking the actual identity of the returned student reveals the error.

How did edge case tests help

Boundary tests exposed the strict comparisons at age 18 and mark 50, which ordinary tests using age 20 and mark 80 would miss. Empty-list tests exposed division and indexing failures. Negative values and out-of-range discounts revealed calculations that completed successfully but violated the intended rules.

Why is a silent wrong result more dangerous than a crash

A crash gives an immediate sign that processing failed. A silent wrong result can look valid, enter reports or later calculations, and lead to incorrect decisions before anyone notices. For example, returning Ali for a search for a missing student can attach information to the wrong person without an error message.

Final verification

The test run used Python 3.12.14. Task 1 passed 26/26 patched checks and Task 2 passed 31/31 corrected checks, for a total of 57/57. Tests include every example input listed in the handout, at least three bad inputs per Task 1 function, and at least three edge-focused tests per Task 2 function. Both valid behavior and rejected inputs were checked using executable assertions.

The tables report observed execution rather than predicted output. No Task 2 row marked No indicates a defect for that particular input; a function can pass normal cases while failing boundary cases. Validation ranges for marks and nonnegative amounts are explicit rules used consistently throughout this submission.

Source

Lab Task 8.pdf, SCD Lab Task 8, IMSciences Peshawar, designed by Engr. Muhammad Umer Haroon, pages 1 to 6.